

Roadmap complet pentru GenAI Engineer

Curriculum riguros: RAG, agenți, document intelligence, fine-tuning, evaluare, securitate și producție

Scopul documentului

Acest curriculum răspunde la o întrebare practică: ce trebuie să înveți pentru a putea realiza, de la descoperirea problemei până la operarea în producție, sistemele descrise de un GenAI Developer care lucrează cu RAG, agenți cu tool use, procesare de documente, fine-tuning local, evaluări, golden datasets, chatbots și guardrails.

Documentul este orientat spre rolul de **AI Engineer / GenAI Engineer**, nu spre cercetare academică pură. Ținta nu este să antrenezi un foundation model de la zero, ci să poți combina modele existente, date private, backend, securitate, cloud și evaluări într-un produs fiabil.

Curriculumul a fost documentat în august 2026. Afirmățiile tehnice importante sunt ancorate în documentații oficiale și lucrări originale, marcate [S1]–[S42]. Bibliografia se află la final.

Rezultatul urmărit

La final trebuie să poți spune, cu dovezi în portofoliu:

- pot transforma o problemă ambiguă de business într-o ipoteză testabilă și într-un contract de succes;
- pot proiecta un sistem AI cu limite, fallback-uri și control uman explicite;
- pot construi ingestion, retrieval, generation, tool use și fine-tuning fără să ascund logica în framework-uri;
- pot crea un golden dataset și pot măsura separat retrievalul, răspunsul, siguranța, costul și latența;
- pot aplica identitatea utilizatorului și permisiunile documentelor end-to-end;
- pot livra API-uri, workers, queues, observabilitate, CI/CD și deployment cloud;
- pot explica stakeholderilor ce poate, ce nu poate și cât costă sistemul.

Profilul profesional real din spatele postării

Lista din postare descrie o intersecție de competențe, nu o singură bibliotecă:

Axă	Ce trebuie să demonstrezi	Pondere orientativă
Software engineering	Backend, API, baze de date, auth, queues, testare, deployment	30%
LLM și ML aplicat	Transformers, embeddings, prompting, fine-tuning, inferență	20%
Data și retrieval	Parsare, chunking, indexare, search, reranking, lineage	20%
Evaluare și siguranță	Golden sets, metrici, red teaming, guardrails, RBAC	20%
Product și comunicare	Discovery, KPI, UX, cost, stakeholder management	10%

Un chatbot care trimite un prompt la un API acoperă doar o fracțiune din rol. Dificultatea reală apare la date murdare, permisiuni, evaluarea regresiiilor, execuții repetate, costuri, atacuri și așteptări de business.

Cum folosești curriculumul

Ritm recomandat

Planul de bază este de **40 de săptămâni**, la 12–15 ore pe săptămână. Dacă ai deja experiență serioasă de backend, poți comprima primele patru săptămâni. Nu comprima evaluarea, securitatea sau producția: acestea sunt exact zonele care separă un demo de un sistem profesional.

Distribuția unei săptămâni:

- 3 ore pentru concepte și documentație primară;
- 7–9 ore pentru implementare;
- 2 ore pentru teste și evaluări;
- 1 oră pentru arhitectură, jurnal de decizii și comunicare;
- 1 oră pentru citirea unui paper sau a unei implementări de referință.

Regula de progres

Nu considera un subiect învățat fiindcă ai urmărit un curs. Pentru fiecare capitol ai nevoie de patru tipuri de dovadă:

- 1 **Explicație:** îl poți explica fără jargon inutil.
- 2 **Implementare:** îl poți construi într-o versiune minimă fără un framework care ascunde pașii.
- 3 **Măsurare:** poți defini și calcula o metrică relevantă.
- 4 **Diagnostic:** poți identifica o defecțiune și izola cauza.

Diagnostic inițial

Înainte să începi, verifică dacă poți realiza fără tutorial:

- un API autentificat cu trei endpoint-uri și validare de schemă;
- o migrare PostgreSQL și o tranzacție corectă;
- un worker asincron care poate relua o sarcină fără dublarea rezultatului;
- teste unitare, integrare și end-to-end;
- o imagine Docker non-root și un pipeline CI;
- o explicație pentru precision, recall, overfitting și data leakage;
- un script Python tipizat, împachetat și configurat prin environment variables.

Orice punct ratat devine obligatoriu în capitolele 1–3.

Capitolul 1 — Inginerie software pentru sisteme AI

1.1 Python modern

Python este limbajul dominant al ecosistemului ML. Chiar dacă backendul principal este Java, C# sau TypeScript, trebuie să poți scrie servicii și pipeline-uri Python curate.

Învăță:

- sintaxă, comprehensions, iterators, generators și context managers;
- type hints, **Protocol**, generics, **TypedDict** și verificare statică;
- **dataclasses** și Pydantic pentru contracte de date;
- excepții, error taxonomy și erori recuperabile versus nerecuperabile;
- **asyncio**, coroutines, tasks, cancellation, timeouts și task groups [S1];
- procese versus threads versus coroutines și efectul GIL;
- packaging cu **pyproject.toml**, environments și dependency locking;
- configurare, secrets și separarea dev/test/prod;
- profiling de CPU și memorie;
- stil, linting și pre-commit hooks.

Exercițiu: implementează un client asincron pentru un model, cu semaphore pentru concurență, timeout, retry cu backoff și anulare corectă.

1.2 API-uri și servicii

Învăță:

- HTTP, REST, status codes, headers, content negotiation și streaming;
- OpenAPI și contract-first development;

- FastAPI: dependency injection, validation, async endpoints, middleware, streaming și testing [S2];
- Django: ORM, migrations, admin, authentication, permissions și RBAC [S3];
- WebSockets și Server-Sent Events pentru răspunsuri în flux;
- versionarea API-urilor și compatibilitatea înapoi;
- idempotency keys pentru operații lungi;
- rate limiting și request size limits;
- separarea controller/service/repository/adapters.

Nu trebuie să alegi un singur framework pentru tot. Django este util când ai un produs business cu utilizatori, roluri, admin și modele relaționale bogate. FastAPI este potrivit pentru servicii AI, ingestion și inferență cu contracte explicite.

1.3 PostgreSQL și acces la date

Învăță:

- modelare relațională, normalizare și denormalizare intenționată;
- indexes, query plans și **EXPLAIN ANALYZE**;
- transactions, isolation levels, locks și race conditions;
- JSONB și când nu trebuie folosit;
- full-text search cu **tsvector**, **tsquery**, ranking și GIN [S14];
- row-level security și politica default-deny [S4];
- migrations compatibile cu deployment fără downtime;
- connection pooling;
- audit trail și soft deletion acolo unde cerințele o impun.

Exercițiu: creează o aplicație multi-tenant în care aceeași interogare returnează rânduri diferite în funcție de identitatea utilizatorului, folosind RLS și teste negative.

1.4 Workers, queues și fiabilitate

Învăță:

- producer, broker, consumer și delivery semantics;
- at-most-once, at-least-once și de ce „exactly once” este de obicei o proprietate compusă;
- idempotency, deduplication keys și transactional outbox;
- retries cu exponential backoff și jitter;
- dead-letter queue și poison messages;
- ordering, partition keys și competing consumers;
- backpressure și limite de concurență;
- job status, heartbeat, leases și recovery;

- saga/compensating actions pentru operații parțiale.

Azure recomandă procesare idempotentă deoarece mesajele pot fi livrate din nou și recomandă dead-letter queues pentru erori persistente [S35][S36].

1.5 Testare

Stăpânește:

- unit tests pentru logică pură;
- integration tests pentru DB, queue, blob storage și model gateway;
- contract tests pentru API-uri și tools;
- end-to-end tests pentru traseul complet;
- property-based testing pentru parsare și validare;
- test doubles: fake, stub și mock, utilizate cu discernământ;
- deterministic fixtures și time freezing;
- load testing și soak testing;
- fault injection: timeout, răspuns invalid, duplicate message, model indisponibil.

În sistemele AI, testele deterministe și eval-urile probabilistice trebuie să coexiste. Nu folosi un LLM-as-judge ca înlocuitor pentru o aserțiune SQL sau pentru verificarea unei permisiuni.

Dovada de stăpânire

Construiește un serviciu de ingestion care primește fișiere, creează un job, publică un mesaj, procesează asincron, persistă rezultatul și expune status. Demonstrează prin teste că repetarea aceleiași cereri sau livrarea duplicată a mesajului nu produce înregistrări duplicate.

Capitolul 2 — Matematică, statistică și ML clasic

2.1 Matematica minimă necesară

Învață suficient cât să înțelegi mecanismele și compromisurile:

- vectori, matrice, produs scalar și multiplicare matricială;
- norme L1/L2, distanță euclidiană și cosine similarity;
- rangul unei matrice și intuiția din spatele LoRA;
- probabilități condiționate și Bayes;
- distribuții, medie, varianță, percentiles și confidence intervals;
- derivată, gradient și chain rule;
- funcții de pierdere, gradient descent, learning rate și regularizare;
- softmax, cross-entropy și log-likelihood.

Nu ai nevoie de demonstrații formale pentru toate, dar trebuie să poți calcula exemple mici și să interpretezi rezultatele.

2.2 Procesul ML

Învăță:

- definirea unei ținte și alegerea unei reprezentări;
- train/validation/test și split după timp, utilizator sau entitate;
- baseline simplu înaintea modelului complex;
- data leakage și contamination;
- underfitting și overfitting;
- feature engineering și pipelines reproductibile;
- class imbalance și threshold selection;
- cross-validation și limitele sale [S7];
- error analysis pe cohorte;
- reproducibilitate: seeds, versiuni de date, cod și environment.

2.3 Metrici

Stăpânește:

- confusion matrix;
- accuracy, precision, recall, specificity și F1;
- micro, macro și weighted averaging;
- ROC-AUC și PR-AUC;
- calibration și probabilități bine calibrate;
- MAE, RMSE și metrice de ranking;
- precision@k, recall@k, hit rate, MRR și nDCG;
- tradeoff între false positives și false negatives în context business.

Exercițiu: construiește un clasificator de documente cu un baseline TF-IDF. Compară-l cu un model de embeddings. Nu raporta doar scorul global; raportează erorile pe categorie și intervale de încredere.

2.4 Data engineering pentru ML

Învăță:

- schema și validarea dataseturilor;
- provenance și lineage;
- deduplicare exactă și near-duplicate;
- label quality, inter-annotator agreement și adjudication;
- sampling și acoperirea cazurilor rare;
- versionarea dataseturilor;

- train-serving skew;
- protecția PII și reguli de retenție.

Dovada de stăpânire

Livrează un raport reproductibil care compară minimum două baseline-uri, justifică split-ul, identifică leakage posibil și explică de ce metrica principală reflectă costul real al erorilor.

Capitolul 3 — Deep learning, NLP și Transformers

3.1 PyTorch practic

Parcurge fluxul complet PyTorch: tensors, datasets și dataloaders, model, autograd, optimization loop și salvare/încărcare [S8].

Învață:

- tensor shapes, broadcasting și device placement;
- forward pass, loss, backward pass și optimizer step;
- batching, padding și attention masks;
- mixed precision și gradient accumulation;
- checkpoints și reluarea antrenării;
- train/eval mode;
- GPU memory: parameters, gradients, optimizer states, activations;
- distributed training doar conceptual la început.

Exercițiu: antrenează o rețea mică de clasificare text și scrie manual bucla de training înainte să folosești **Trainer**.

3.2 NLP esențial

Învață:

- normalizare Unicode și efectele asupra limbii române;
- word, subword și byte-level tokenization;
- BPE, WordPiece și SentencePiece conceptual;
- vocabulary, unknown tokens și special tokens;
- truncation, padding și alignment;
- language modeling autoregresiv;
- pretraining, instruction tuning și preference tuning, la nivel conceptual.

Pipeline-ul unui tokenizer include normalizare, pre-tokenizare, model și post-procesare [S10]. Verifică explicit diacriticele, tabelele și textele multilingve.

3.3 Arhitectura Transformer

Citește lucrarea originală „Attention Is All You Need” [S9] și înțelege:

- embeddings poziționale;
- query, key și value;
- scaled dot-product attention;
- multi-head attention;
- feed-forward layers, residual connections și layer normalization;
- causal mask versus bidirectional attention;
- encoder-only, decoder-only și encoder-decoder;
- parametri versus activations;
- context window și complexitatea attention;
- autoregressive decoding și KV cache [S11].

Trebuie să poți explica de ce generarea token-cu-token are alt profil de latență decât embedding-ul unui batch și de ce KV cache economisește recomputare, dar consumă memorie.

3.4 Comportamentul LLM

Învăță:

- system, developer, user și tool messages ca priorități definite de provider;
- sampling: temperature, top-p, greedy și beam search;
- context limits și lost-in-the-middle;
- factualitate versus fluentă;
- hallucination ca termen umbrelă, nu diagnostic suficient;
- prompt sensitivity și nondeterminism;
- model selection pe calitate, cost, latență, context, limbi și safety;
- closed-weight API versus open-weight self-hosted;
- quantization și efectele asupra memoriei și calității;
- batch inference, continuous batching și throughput.

Dovada de stăpânire

Implementează un notebook educațional care vizualizează tokenizarea, attention masks, top-k sampling și costul aproximativ al KV cache. Explică rezultatele într-un README destinat unui backend engineer.

Capitolul 4 — Integrarea modelelor și prompting robust

4.1 Model gateway

Nu împrăștia apelurile către provider prin aplicație. Construiește un model gateway care centralizează:

- configurarea modelelor și versiunea lor;
- timeouts, retries și circuit breaker;
- rate limits și concurrency limits;
- cost accounting pe request, user și feature;
- logging cu redacția datelor sensibile;
- fallback controlat;
- caching numai când semantica permite;
- capturarea request ID și model metadata;
- integrarea modelelor locale și remote prin aceeași interfață.

4.2 Prompt design

Învăță:

- instrucțiuni clare, delimitarea datelor și exemple few-shot;
- prompt templates versionate;
- context construction și ordonarea informației;
- decompoziția sarcinii;
- self-checks cu limite explicite;
- cererea de abstention când datele nu susțin răspunsul;
- prompt chaining versus un prompt monolitic;
- schimbarea promptului tratată ca schimbare de cod și evaluată.

Nu memora „prompt tricks”. Formulează un contract: intrare, ieșire, criterii, excepții și sursa adevărului.

4.3 Structured outputs și tool schemas

Învăță:

- JSON Schema;
- Pydantic models;
- enumerations, required fields și **additionalProperties**;
- parsing, validation și repair limitat;
- diferența dintre „JSON valid” și „valoare corectă semantic”;
- versionarea tool contracts;
- delimitarea argumentelor modelului de autorizația operației.

Exercițiu: cere modelului să extragă o structură complexă din 200 de exemple. Măsoară schema-valid rate, field accuracy și abstention accuracy. Nu declara succes dacă doar parserul nu aruncă excepție.

4.4 Managementul contextului

Învăță:

- token budgeting per componentă;
- truncation determinist și prioritizarea contextului;
- sliding window și summaries cu verificarea pierderii de informație;
- memorie conversațională versus memorie de business;
- politica de retenție;
- cache invalidation;
- separarea instrucțiunilor de conținutul neîncrezut.

Dovada de stăpânire

Livrează un gateway cu două modele, structured output, retry numai pentru erori recuperabile, buget de cost și traces. Un test trebuie să demonstreze comportamentul când modelul întoarce JSON invalid, când depășește timeout-ul și când providerul aplică rate limiting.

Capitolul 5 — Document intelligence și ingestion

5.1 Anatomia documentelor

Învăță diferențele dintre:

- PDF digital, PDF scanat și PDF cu layer OCR;
- ordinea vizuală versus ordinea de citire;
- paragrafe, titluri, liste, footnotes și captions;
- tabele cu merged cells și tabele pe mai multe pagini;
- formule, imagini și grafice;
- DOCX, HTML, email și attachments;
- encoding, Unicode și language detection.

Un PDF nu este o bază de date de paragrafe. Este adesea o colecție de instrucțiuni grafice. Orice pipeline serios trebuie să păstreze legătura dintre text, pagină, bounding box și fișierul original.

5.2 Parsing și OCR

Învăță:

- parsere native pentru PDF/DOCX/HTML;
- OCR și confidence scores;

- layout analysis;
- GROBID pentru articole științifice și TEI XML;
- parsarea referințelor bibliografice și rezolvarea DOI;
- Azure Document Intelligence pentru layout, tabele și bounding regions [S17];
- fallback-uri între parsere;
- detectarea paginilor sau câmpurilor cu confidence scăzut.

GROBID este potrivit pentru structura lucrărilor științifice, metadata și referințe [S16]. Pentru facturi, formulare sau documente generale, un serviciu de layout/document intelligence poate fi mai potrivit.

5.3 Modelul intern al documentului

Definește un model canonic cu:

- **document_id**, **source_id**, hash și versiune;
- tenant, ACL și classification;
- mime type, limbă și timestamps;
- elemente structurale și ierarhia secțiunilor;
- pagină, coordonate și ordinea de citire;
- text brut, text normalizat și text folosit la embeddings;
- legături dintre tabele, captions și footnotes;
- provenance pentru fiecare transformare.

5.4 Deduplicare și versionare

Învăță:

- content hashes pentru duplicate exacte;
- canonicalization;
- MinHash/SimHash sau embeddings pentru near-duplicates;
- diferența dintre versiune nouă și document nou;
- deletion propagation către indexuri și cache;
- reprocessing controlat când se schimbă parserul sau embedderul;
- lineage între blob, parse, chunks și index entries.

5.5 Pipeline asincron

Proiectează stări explicite:

```
RECEIVED -> VALIDATED -> STORED -> PARSED -> ENRICHED -> INDEXED -> READY
      \-> QUARANTINED
      \-> FAILED_RETRYABLE
      \-> FAILED_FINAL
```

Pentru fiecare etapă definește input, output, idempotency key, timeout, retry policy, metrici și mecanism de compensare.

Dovada de stăpânire

Procesează un corpus mixt de minimum 100 de documente, incluzând scanări și tabele. Creează un raport cu parse success, OCR confidence, table extraction accuracy, duplicate rate și eșantioane inspectate manual.

Capitolul 6 — Information retrieval și RAG

6.1 De ce RAG

Lucrarea RAG combină memoria parametrică a modelului cu o memorie externă recuperată la cerere și evidențiază avantajele pentru actualizarea cunoașterii și provenance [S12]. În aplicații enterprise, RAG este în primul rând un sistem de retrieval și control al contextului; LLM-ul este ultima etapă.

Folosește RAG când informația:

- este privată, actualizată frecvent sau trebuie citată;
- încapă într-un corpus interogabil;
- trebuie filtrată după utilizator;
- poate fi reprezentată prin fragmente relevante.

Nu folosi RAG ca remediu universal pentru reguli de business deterministe, calcule exacte sau acțiuni care ar trebui implementate prin cod și tools.

6.2 Embeddings

Învăță:

- dense vector representations;
- cosine, inner product și L2;
- normalizarea vectorilor;
- bi-encoder pentru retrieval și cross-encoder pentru scoring pereche;
- modele multilingve și domain shift;
- dimensiune, cost de stocare și throughput;
- batch embedding și rate limits;
- versionarea embedderului și reindexarea;
- evaluarea pe propriul corpus, nu doar pe leaderboard.

SBERT produce sentence embeddings comparabile prin cosine similarity și face căutarea semantică practică la scară [S13].

6.3 Chunking

Experimentează cu:

- fixed tokens cu overlap;
- sentence-based;
- section-aware;
- parent-child;
- table-aware;
- semantic chunking;
- chunks specializate pentru cod sau policy documents.

Păstrează titlul, calea secțiunii, pagina, ACL și legătura la elementul original. Overlap-ul excesiv crește costul, duplică rezultate și poate distorsiona rankingul.

6.4 Lexical, semantic și hybrid search

Învăță:

- inverted index și BM25 conceptual;
- PostgreSQL full-text search [S14];
- exact versus approximate nearest neighbor;
- HNSW și IVFFlat conceptual;
- reciprocal rank fusion;
- metadata filters;
- pre-filter versus post-filter;
- efectul filtrelor asupra recall-ului ANN;
- monitorizarea recall-ului aproximativ față de exact search.

pgvector documentează HNSW, IVFFlat, filtering și recomandă măsurarea recall-ului prin comparație cu exact search [S15].

6.5 Query processing

Învăță:

- normalization și language detection;
- query classification;
- intent routing;
- query rewriting;
- expansion și decomposition pentru întrebări multi-hop;
- filters extrase din întrebare;
- conversational query resolution;
- HyDE doar ca experiment evaluat, nu default automat.

6.6 Reranking și context assembly

Pipeline recomandat pentru învățare:

```
query
-> lexical candidates
-> dense candidates
-> fusion + dedup
-> ACL filter
-> cross-encoder reranker
-> diversity/coverage
-> context budget
-> generation with citations
```

Învăță:

- cross-encoder reranking;
- top-k și candidate pool size;
- Maximal Marginal Relevance conceptual;
- diversity versus redundanță;
- context ordering;
- evidence sufficiency;
- citation mapping la sursa exactă;
- răspuns „nu există suficiente informații”.

6.7 RAG cu acces controlat

Permisunile nu sunt un filtru cosmetic după generare. Aplică-le înainte ca textul să intre în context:

- identitate verificată;
- tenant și grupuri propagate;
- ACL stocat cu fiecare document/chunk;
- filtrare în lexical și vector search;
- RLS sau alt enforcement în storage [S4];
- citări și cache partitionate după principal;
- index deletion la revocarea accesului;
- teste de non-interferență între utilizatori.

Dovada de stăpânire

Construiește un RAG fără framework în prima iterație: parser, chunker, embeddings, PostgreSQL full-text + pgvector, fusion, reranker și prompt. Apoi poți adopta un framework, păstrând metricile și contractele.

Capitolul 7 — Evaluare, golden datasets și experimentare

7.1 De ce evaluarea este o componentă de produs

Un sistem GenAI nu este „bun” în general. Este acceptabil pe o distribuție de sarcini, pentru un risc și un cost definite. Fără dataset și criterii, schimbarea modelului, promptului, chunkingului sau rerankerului este o opinie.

Separă cel puțin cinci straturi:

- 1 calitate datelor și parsingului;
- 2 retrieval;
- 3 generation;
- 4 siguranță și permisiuni;
- 5 performanță operațională și valoare business.

RAGAS propune evaluarea separată a contextului, folosirii fidele a acestuia și calității răspunsului [S18].

7.2 Construirea unui golden dataset

Pentru fiecare exemplu stochează:

- ID stabil și versiune;
- întrebare/input;
- intenție și categorie;
- documente/chunks relevante sau reguli de relevanță;
- răspuns de referință, când este posibil;
- rubrică de evaluare;
- comportament acceptat de abstention;
- identitatea și permisiunile utilizatorului;
- nivelul de risc;
- sursa exemplului: real, expert, sintetic sau adversarial.

Datasetul trebuie să includă:

- happy paths;
- întrebări ambigue;
- informație absentă;
- contradicții între documente;
- documente vechi și noi;
- tabele și multi-hop;
- typo, limbi multiple și formulări rare;
- prompt injection direct și indirect;
- utilizatori fără drepturi;

- inputuri foarte lungi;
- tool errors și timeouts.

Începe cu 100–200 exemple curate. Crește din incidente reale și feedback, nu doar prin generare sintetică.

7.3 Metrici pentru retrieval

Învăță și implementează:

- hit rate@k;
- recall@k;
- precision@k;
- MRR;
- nDCG;
- coverage pe categorii;
- latency p50/p95/p99;
- index size și cost per query.

Evaluează lexical, dense, hybrid și reranked separat. Un răspuns bun nu dovedește că retrievalul este bun: modelul poate răspunde din parametri sau din întâmplare.

7.4 Metrici pentru generation

Folosește o combinație:

- exact match sau execution match pentru ieșiri deterministe;
- schema validity și field-level accuracy;
- answer correctness pe rubrică;
- faithfulness/groundedness;
- citation precision și citation completeness;
- completeness;
- abstention precision/recall;
- stil și format numai după corectitudine;
- human preference pentru experiențe deschise.

7.5 LLM-as-judge

Poate scala evaluarea, dar nu este adevăr absolut. Lucrarea MT-Bench identifică bias de poziție, verbosity și self-enhancement [S19].

Practici obligatorii:

- rubrică specifică și exemple ancoră;
- separarea criteriilor în locul unui scor vag;
- randomizarea ordinii pentru comparații pereche;

- judge diferit de modelul evaluat când este posibil;
- calibrare pe un eșantion evaluat de oameni;
- reevaluarea judge-ului când îi schimbă versiunea;
- păstrarea outputului și justificării pentru audit;
- metrice deterministe acolo unde acestea există.

7.6 Experimentare și regression gates

Pentru fiecare experiment versionează:

- model și parametri;
- prompt;
- embedder;
- chunker;
- retrieval parameters;
- reranker;
- dataset;
- cod și environment;
- cost, latență și scoruri;
- exemple câștigate și pierdute.

Definește gates, de exemplu:

- retrieval recall@10 nu scade cu peste 1 punct;
- niciun test de access control nu eșuează;
- faithfulness crește sau rămâne în interval;
- p95 latency rămâne sub buget;
- costul mediu nu depășește pragul;
- regresile pe categoriile cu risc ridicat blochează release-ul.

Dovada de stăpânire

Compară două versiuni complete ale sistemului și publică un raport care arată distribuții, confidence intervals, costuri, latențe și minimum 20 de exemple analizate manual.

Capitolul 8 — Agenți, tool use și workflows

8.1 Definiții operaționale

Un **workflow** are trasee proiectate explicit. Un **agent** lasă modelul să decidă dinamic următorul pas sau instrument. ReAct intercalează reasoning și actions pentru interacțiunea cu surse externe [S20], iar Toolformer studiază decizia modelului privind când și cum să apeleze API-uri [S21].

Regula de proiectare: folosește cod determinist pentru reguli cunoscute și acordă autonomie numai acolo unde selecția semantică aduce valoare măsurabilă.

8.2 Tool design

Un tool bun are:

- nume și descriere neambiguous;
- schemă strictă;
- input mic și semantic clar;
- rezultat structurat;
- erori tipizate;
- timeout și cancellation;
- idempotency pentru operații mutabile;
- scope de permisiuni minim;
- audit log;
- confirmare pentru acțiuni ireversibile sau cu impact.

Modelul poate propune un tool call. Aplicația trebuie să autorizeze și să execute. Nu transforma modelul în security boundary.

8.3 State și orchestration

Învăță:

- finite-state machines și graphs;
- state serializabil;
- checkpoints;
- durable execution;
- retries pe nod, nu pe întregul workflow;
- compensating actions;
- parallel branches și joins;
- budgets pentru pași, tokens, timp și bani;
- termination conditions și loop detection;
- pause/resume și human-in-the-loop.

LangGraph documentează durable execution, persistence și human-in-the-loop pentru agenți stateful [S22]. Învăță însă conceptele înainte să depinzi de implementarea sa.

8.4 Memorie

Separă:

- working state al execuției;
- short-term conversation memory;

- user profile/preferences;
- enterprise knowledge prin RAG;
- audit history.

Definește cine scrie, cine citește, cât timp se păstrează și cum se șterge fiecare tip. Nu salva automat afirmațiile modelului ca „fapte” despre utilizator.

8.5 Human-in-the-loop

Folosește approval gates pentru:

- trimiterea unui mesaj extern;
- modificarea sau ștergerea datelor;
- execuția SQL mutabil;
- accesul la date foarte sensibile;
- costuri peste prag;
- cazuri cu confidence scăzut;
- decizii care afectează persoane.

Interfața de aprobare trebuie să arate acțiunea, argumentele, sursa datelor, efectul și opțiunea de editare sau respingere.

8.6 Evaluarea agenților

Măsoară:

- task success rate;
- tool selection accuracy;
- argument correctness;
- număr de pași;
- loop/failure rate;
- recovery după tool error;
- unauthorized action rate;
- approval precision;
- cost și latență pe task;
- trace completeness.

Dovada de stăpânire

Construiește un agent cu minimum trei tools, state persistent, limită de pași, două approval gates și fault injection. Compară-l cu un workflow determinist pe același golden set și justifică unde autonomia merită costul și riscul.

Capitolul 9 — Fine-tuning local și text-to-SQL

9.1 Când folosești fine-tuning

Fine-tuning este potrivit pentru:

- stil sau format stabil;
- transformări repetabile;
- clasificare și extraction;
- comportament de tool calling;
- adaptare de domeniu evaluată;
- reducerea promptului sau folosirea unui model mai mic.

Nu îl folosi ca primă soluție pentru cunoaștere privată care se schimbă. Pentru aceasta, RAG sau tools sunt de regulă mai ușor de actualizat și auditat.

Înainte de fine-tuning construiește:

- 1 baseline zero/few-shot;
- 2 golden test set înghețat;
- 3 error taxonomy;
- 4 dataset cu proveniență și licență;
- 5 buget de compute și criteriu de oprire.

9.2 Dataset pentru SFT

Învăță:

- instruction/input/output și chat templates;
- tokenizer compatibil cu modelul;
- masking pentru loss;
- lungimea secvenței și packing;
- curățare, dedup și near-duplicate detection;
- train/validation/test fără leakage de schemă sau template;
- balansarea categoriilor;
- exemple negative și refuzuri;
- date sintetice validate;
- data cards și versionare.

Calitatea și diversitatea datasetului contează mai mult decât acumularea de exemple aproape identice.

9.3 LoRA și QLoRA

LoRA îngheață modelul de bază și antrenează matrice low-rank, reducând puternic numărul parametrilor antrenabili [S23]. QLoRA propagă gradientul printr-un model de bază quantizat la 4 biți către adaptoare LoRA [S24].

Învată:

- rank, alpha, dropout și target modules;
- full fine-tuning versus PEFT;
- 4-bit quantization, NF4 și double quantization conceptual;
- learning rate, batch size și gradient accumulation;
- eval/save strategy și early stopping;
- merge versus adapter separat;
- incompatibilități de tokenizer sau chat template;
- catastrophic forgetting și overfitting;
- licența modelului și a datasetului;
- PEFT și Trainer din ecosistemul Hugging Face [S25][S26].

9.4 Experiment tracking

Înregistrează:

- base model și revision;
- dataset hash;
- hyperparameters;
- seed și environment;
- losses și eval metrics;
- GPU, timp și cost;
- checkpoints;
- adapter artifact;
- commit hash;
- raportul de comparație cu baseline.

MLflow Tracking poate stoca parametri, versiuni de cod, metrice și artifacts pentru fiecare run [S38].

9.5 Serving local

Învată:

- model loading și device mapping;
- quantized inference;
- batching și continuous batching;

- KV cache;
- throughput versus time-to-first-token;
- concurrency și memory limits;
- vLLM sau un server echivalent;
- health/readiness probes;
- model warm-up;
- rolling/canary deployment;
- fallback la un model remote.

9.6 Text-to-SQL

Spider este un benchmark clasic cross-domain pentru semantic parsing și text-to-SQL [S28]. Pentru un sistem HR real, învață:

- schema linking;
- dialect și semantic layer;
- join paths și business definitions;
- few-shot retrieval pe exemple similare;
- constrained generation;
- parsing SQL într-un AST;
- allowlist pentru operații, tabele, coloane și funcții;
- read-only credentials și statement timeout;
- query cost estimation și row limit;
- sandbox database;
- execution accuracy, nu doar string exact;
- result validation și explicație;
- protecția împotriva extragerii datelor neautorizate.

Nu executa SQL generat cu credențiale privilegiate. Permisunile bazei de date, RLS și un validator determinist trebuie să rămână active chiar dacă modelul pare sigur.

Dovada de stăpânire

Fine-tune un model open-weight de dimensiune fezabilă prin QLoRA pentru o schemă HR fictivă. Compară base, few-shot/RAG și fine-tuned pe un test set complet separat. Raportează valid SQL, execution accuracy, forbidden-query rate, latență și memorie.

Capitolul 10 — Guardrails, securitate și guvernanta

10.1 Threat modeling pentru LLM applications

OWASP Top 10 pentru aplicații LLM include prompt injection, sensitive information disclosure, supply chain, data/model poisoning, improper output handling, excessive agency, system prompt leakage, vector/embedding weaknesses, misinformation și unbounded consumption [S29].

Modelează activele, actorii, trust boundaries și fluxurile:

- user input;
- uploaded documents;
- retrieved content;
- model/provider;
- tools și downstream APIs;
- vector store și memory;
- logs și evaluation datasets;
- admin interfaces;
- CI/CD și model artifacts.

10.2 Prompt injection

Învăță:

- direct versus indirect injection;
- jailbreak versus injection orientat spre aplicație;
- instrucțiuni malițioase în PDF, web, email sau tool output;
- exfiltration prin tool calls sau output;
- payload encoding și obfuscation;
- multi-turn persistence;
- de ce delimitarea promptului nu este o garanție.

Defense in depth:

- tratează conținutul recuperat ca date neîncredute;
- separă data plane de control plane;
- minimizează tools și permisiunile;
- validează argumentele și outputurile;
- filtrează surse și tipuri de conținut;
- approval pentru acțiuni sensibile;
- sandbox pentru cod și SQL;
- canary tokens și anomaly detection;
- red-team dataset la fiecare release;

- limitează impactul chiar dacă modelul este compromis.

10.3 Llama Guard și clasificatoare safety

Llama Guard este un model de safeguard pentru clasificarea inputului și outputului conversațional [S31]. Învăț să-l folosești ca un strat, nu ca unicul control:

- taxonomy adaptată politicii aplicației;
- input moderation și output moderation;
- praguri și costul false positives/negatives;
- evaluare multilingvă și pe domeniu;
- versionarea politicii;
- fallback și review uman;
- observarea evaziunilor.

Un safety classifier nu înlocuiește RBAC, validatorul de tools, sandboxing sau rate limits.

10.4 Identity și access control

Învăț:

- OAuth 2.0 și OpenID Connect;
- delegated versus application permissions;
- service principals și managed identities;
- RBAC, ABAC și RLS;
- least privilege;
- token audience, issuer, scopes și expiration;
- group overage și caching atent al permisiunilor;
- audit log și access reviews.

Microsoft Graph distinge accesul delegat, în numele unui utilizator, de accesul app-only [S33]. Pentru un assistant Teams cu SharePoint, păstrează această diferență vizibilă în arhitectură și în teste. Teams bots pot folosi OAuth 2.0 și SSO pentru acces în numele utilizatorului [S34].

10.5 Privacy și data governance

Învăț:

- data classification;
- PII detection și redaction;
- encryption in transit/at rest;
- key și secret management;
- data residency;
- retention și deletion;
- consent și purpose limitation;

- log minimization;
- vendor data-use settings și contractual boundaries;
- dreptul de a șterge și propagarea în indexuri, cache și backups conform politicii.

10.6 Responsible AI

NIST AI RMF și profilul pentru GenAI oferă un cadru pentru map, measure, manage și govern riscurile [S30]. Învăță:

- impact assessment;
- transparență și limitations;
- fairness și evaluare pe subgrupuri relevante;
- human oversight;
- incident response;
- model/system cards;
- ownership și risk acceptance;
- monitorizare post-deployment.

Dovada de stăpânire

Pentru fiecare proiect scrie un threat model și rulează o suită adversarială. Demonstrează că un document cu prompt injection nu poate declanșa un tool privilegiat și că un utilizator nu poate recupera sau cita conținut fără drepturi.

Capitolul 11 — Cloud, MLOps, LLMOps și producție

11.1 Containerizare și environments

Învăță:

- Dockerfiles reproducibile și multi-stage builds;
- imagini mici, pinned dependencies și vulnerability scanning;
- rulare non-root;
- Docker Compose pentru mediul local;
- config versus secrets;
- dev/test/staging/prod parity;
- software bill of materials;
- CPU/GPU runtime differences.

11.2 Arhitectura de referință

Pentru un sistem enterprise AI, separă:

Client / Teams

- > API + Identity
- > Orchestrator / Model Gateway
- > Retrieval + SQL Tools
- > Model API sau Model Server

Upload

- > Blob Storage
- > Queue
- > Parsing/OCR Workers
- > Canonical Store
- > Search Indexes

Toate componentele

- > Traces + Metrics + Logs + Cost
- > Evaluation Store + Feedback

Azure Well-Architected recomandă adaptarea arhitecturii AI la obiective business, constrângeri tehnice și risk posture [S42].

11.3 Azure practic

Învăță suficient pentru proiectele din postare:

- resource groups, subscriptions și IAM;
- Blob Storage și lifecycle policies;
- Queue Storage versus Service Bus;
- Azure Functions, Container Apps sau AKS și criteriile de alegere;
- Azure Database for PostgreSQL;
- Key Vault și managed identities;
- Azure AI Document Intelligence;
- networking, private endpoints și egress controls;
- monitoring și cost management;
- Infrastructure as Code cu Terraform sau Bicep.

Nu trebuie să memorezi portalul. Trebuie să poți reproduce mediul prin cod și să explici trust boundaries.

11.4 CI/CD și release engineering

Pipeline minim:

- 1 lint și type check;
- 2 unit tests;
- 3 integration tests cu servicii reale containerizate;
- 4 security/dependency scan;
- 5 build immutable artifact;
- 6 offline evals și safety gates;
- 7 deploy staging;

- 8 smoke și end-to-end tests;
- 9 canary/gradual rollout;
- 10 monitorizare și rollback.

Versionează prompturile, dataseturile, modelul, index schema și tool schemas împreună cu release-ul.

11.5 Observabilitate

OpenTelemetry definește traces, metrics și logs ca semnale complementare [S37]. Pentru o cerere AI urmărește trace-ul complet:

- autentificare;
- query rewrite;
- fiecare retrieval call;
- reranking;
- model request;
- tool call;
- approval;
- response și citations.

Metrici tehnice:

- request rate și error rate;
- p50/p95/p99 latency;
- queue depth și oldest message age;
- parse failures și retries;
- tokens input/output;
- time-to-first-token;
- model/tool timeout rate;
- GPU/CPU/memory;
- cost per request, task, tenant și feature.

Metrici AI:

- retrieval recall proxy și no-result rate;
- groundedness/citation rate;
- abstention rate;
- guardrail trigger rate;
- tool success și unauthorized attempts;
- user correction și escalation rate;
- drift pe categorii.

Nu introduce promptul brut sau PII în logs fără o politică explicită. Folosește sampling, redaction și acces restricționat.

11.6 Scalare și operare

Învață:

- stateless API versus stateful workers;
- horizontal scaling și autoscaling;
- batching și queue-based load leveling;
- Kubernetes Deployments, Jobs și StatefulSets conceptual [S39];
- circuit breakers și bulkheads;
- graceful degradation;
- cache și invalidare;
- multi-region doar dacă cerințele justifică;
- backup/restore și disaster recovery;
- SLO, error budget și runbooks;
- incident postmortems.

Dovada de stăpânire

Deployează un proiect în cloud prin IaC. Include dashboard, alerte, runbook, backup testat, cost estimate și un exercițiu de incident: provider indisponibil, queue backlog sau index corupt.

Capitolul 12 — Stakeholders, product discovery și UX

12.1 Transformarea cererii în problemă

Când auzi „poate AI să facă X?”, nu răspunde imediat cu arhitectura. Clarifică:

- cine face activitatea și pentru cine;
- ce rezultat de business urmărește;
- procesul actual și costul lui;
- volumul, frecvența și variația cazurilor;
- datele disponibile și drepturile asupra lor;
- costul unui răspuns greșit;
- ce decizie sau acțiune urmează outputului;
- cât human review este acceptabil;
- baseline fără AI;
- pragul de succes și condiția de oprire.

12.2 AI suitability

Evaluează fiecare use case pe patru axe:

Axă	Întrebarea decisivă	Semnal de risc
Fezabilitate	Există date și o sarcină pe care o putem evalua?	Nu există exemple reale
Valoare	Economisește timp, bani sau reduce risc?	Este doar un demo atractiv
Toleranță la eroare	Ce se întâmplă când greșește?	Outputul produce automat prejudicii
Operabilitate	Putem monitoriza și corecta sistemul?	Nu există owner sau feedback loop

Preferă augmentation înaintea automatizării totale atunci când consecințele sunt ridicate sau calitatea este instabilă.

12.3 POC, pilot și producție

Definește fazele:

- **Spike tehnic:** verifică o necunoscută izolată.
- **POC:** dovedește fezabilitatea pe date reprezentative.
- **Pilot:** utilizatori reali, volum limitat, feedback și proceduri manuale.
- **Producție:** SLO, securitate, observabilitate, ownership, suport și cost control.

Un notebook poate fi POC. Nu este automat o arhitectură de producție.

12.4 Experiment brief de o pagină

Pentru fiecare inițiativă scrie:

- problem statement;
- user și workflow;
- baseline actual;
- ipoteză;
- input/output și limite;
- golden sample;
- metrică principală și guardrail metrics;
- risc și human oversight;
- cost/timp maxim pentru experiment;
- decizie după rezultat: continue, change sau stop.

12.5 Comunicarea incertitudinii

Înlocuiește promisiunile vagi cu afirmații testabile:

Nu știm încă dacă acoperă toate cazurile HR.

Pe eșantionul curent de 150 de întrebări, baseline-ul obține X.
Principalele eșecuri sunt Y și Z.
Propunem pilot pentru grupul A, cu review uman pentru categoria B.
Oprim lansarea dacă metrica C scade sub pragul D.

12.6 Human-centered AI

Google PAIR recomandă definirea nevoilor utilizatorilor, a succesului, controlului, feedbackului și mental models [S40]. Microsoft HAX oferă ghiduri pentru comportamentul sistemului inițial, în timpul interacțiunii, când greșește și în timp [S41].

Învață să proiectezi:

- onboarding care explică ce poate și cât de bine;
- citări și explicații suficiente pentru decizie;
- editare, confirmare și undo;
- feedback explicit și implicit;
- graceful failure și recovery;
- escaladare către om;
- transparență privind folosirea datelor;
- accessibility și limbi relevante.

12.7 Artefacte de comunicare

Trebuie să poți produce:

- architecture decision record;
- system context diagram;
- data flow și trust boundaries;
- risk register;
- evaluation report;
- model/system card;
- demo cu caz reușit și caz eșuat;
- release note cu schimbări de model/prompt/dataset;
- postmortem fără culpabilizare.

Dovada de stăpânire

Pentru un proiect, simulează un discovery interview, scrie experiment brief, prezintă rezultatele în 10 minute și răspunde la: „de ce nu 100% automat?”, „ce se întâmplă dacă greșește?”, „cât costă?” și „cum știm că versiunea nouă este mai bună?”.

Capitolul 13 — Cele trei proiecte de portofoliu

Proiectul A — Research Assistant pentru articole științifice

Cerințe funcționale

- Django pentru utilizatori, organizații, roluri și admin;
- upload și catalog de articole;
- GROBID pentru metadata, secțiuni și referințe;
- graph de citări;
- keyword + semantic search;
- reranking;
- întrebări cu răspunsuri citate la pagină/secțiune;
- collections și notes;
- audit log.

Cerințe non-funcționale

- ingestion asincron și idempotent;
- RBAC și teste cross-tenant;
- versionarea parserului, chunkerului și embedderului;
- golden dataset de minimum 150 de întrebări;
- raport retrieval/generation separat;
- p95 latency și cost per question;
- threat model pentru document injection;
- deployment reproducibil.

Extensie avansată

Rezolvă DOI, construiește muchii de citare, identifică lucrări retractate sau versiuni și oferă răspunsuri multi-document cu evidence coverage.

Proiectul B — HR Assistant pe Teams și SharePoint

Cerințe funcționale

- Teams bot cu SSO;
- retrieval dintr-un corpus SharePoint simulat sau real autorizat;
- acces delegat și document-level permissions;
- răspunsuri despre politici cu citări;
- text-to-SQL read-only peste o bază HR fictivă;
- tool calls pentru concedii, headcount și organigramă;
- fine-tuned adapter QLoRA pentru text-to-SQL sau routing;
- approval pentru orice acțiune mutabilă.

Cerințe de securitate

- least privilege;
- SQL AST validation, allowlists, row limit și timeout;
- RLS și credentials read-only;
- nicio informație despre alt departament fără acces;
- redaction în logs;
- prompt injection suite;
- audit pentru tool calls și aprobări.

Evaluare

- retrieval recall@k;
- answer faithfulness și citation correctness;
- SQL execution accuracy;
- forbidden-query rate zero pe golden set;
- tool selection și arguments accuracy;
- p95 și cost/task;
- user task completion într-un pilot simulat.

Proiectul C — Document Intelligence Pipeline pe Azure

Arhitectură

- FastAPI pentru upload și status;
- Azure Blob Storage;
- Azure Queue Storage sau Service Bus;
- workers containerizați;
- Azure Document Intelligence pentru layout/OCR/tabele;
- PostgreSQL pentru metadata și rezultate canonice;
- dedup exact și near-duplicate;
- categorizare;
- dead-letter queue și reprocessing controlat;
- OpenTelemetry și dashboards;
- IaC.

Teste de reziliență

- același fișier încărcat de trei ori;
- worker întrerupt după salvare, înainte de acknowledge;
- răspuns parțial sau timeout de la OCR;

- poison document;
- backlog mare;
- schimbarea parser version;
- ștergerea documentului și propagarea în toate store-urile.

Evaluare

- parse success;
- field/table accuracy pe eșantion etichetat;
- dedup precision/recall;
- throughput și queue age;
- cost/document și cost/pagină;
- reprocessing correctness;
- incident recovery time.

Capitolul 14 — Planul de 40 de săptămâni

Faza 1 — Fundamentul de producție, săptămânile 1–4

Săptămâna 1

- Python typing, packaging și Pydantic;
- exerciții cu generators și context managers;
- script CLI pentru validarea documentelor;
- teste și linting.

Săptămâna 2

- **asyncio**, timeouts, cancellation și concurrency limits;
- client model fake cu fault injection;
- măsurarea latenței și concurenței.

Săptămâna 3

- FastAPI, OpenAPI și auth;
- PostgreSQL, migrations și transactions;
- endpoint de creare și citire job.

Săptămâna 4

- queue, worker, idempotency și DLQ;
- Docker Compose și integration tests;
- demo end-to-end ingestion minimal.

Gate: nu avansezi dacă duplicate delivery produce rezultate duplicate sau dacă nu poți explica recovery-ul după crash.

Faza 2 — ML și LLM fundamentals, săptămânile 5–8

Săptămâna 5

- train/validation/test, leakage și baselines;
- precision, recall, F1 și PR curve;
- clasicator TF-IDF.

Săptămâna 6

- PyTorch tensors, datasets, autograd și training loop;
- clasicator neural mic;
- experiment tracking minimal.

Săptămâna 7

- tokenization și Transformer paper;
- vizualizarea attention masks;
- sampling și structured generation.

Săptămâna 8

- model gateway, retries, budgets și traces;
- comparație între două modele pe 50 de exemple;
- raport cost-calitate-latență.

Gate: alegerea modelului trebuie justificată prin date, nu prin reputație.

Faza 3 — Document intelligence, săptămânile 9–12

Săptămâna 9

- anatomia PDF/DOCX/HTML;
- parsere și model canonic;
- corpus de test mixt.

Săptămâna 10

- OCR, layout și table extraction;
- evaluare manuală pe pagini etichetate;
- confidence și quarantine flow.

Săptămâna 11

- GROBID, TEI și bibliographic references;
- canonical IDs și citation graph.

Săptămâna 12

- dedup, versioning, lineage și deletion;
- ingestion complet cu reprocessing;

- raport de calitate.

Gate: orice fragment indexat trebuie urmărit până la fișier, versiune, pagină și parser.

Faza 4 — Retrieval și RAG, săptămânile 13–18

Săptămâna 13

- embeddings, cosine și exact search;
- evaluarea a două modele de embeddings;
- semantic search minimal.

Săptămâna 14

- PostgreSQL FTS și BM25 conceptual;
- benchmark lexical;
- query analysis pe erori.

Săptămâna 15

- pgvector, HNSW/IVFFlat și metadata filters;
- exact versus approximate recall;
- index performance.

Săptămâna 16

- chunking experiments;
- parent-child și section-aware;
- dataset de relevanță.

Săptămâna 17

- hybrid search, fusion și reranking;
- ablation study pentru fiecare componentă.

Săptămâna 18

- generation cu citations și abstention;
- context budgeting;
- Research Assistant v1.

Gate: raportezi retrieval independent de generation și poți arăta de ce fiecare etapă există.

Faza 5 — Evaluare, săptămânile 19–22

Săptămâna 19

- taxonomia utilizatorilor și întrebărilor;
- golden dataset v1 cu minimum 100 exemple;
- guidelines de anotare.

Săptămâna 20

- retrieval metrics și deterministic checks;
- dashboard de comparație.

Săptămâna 21

- faithfulness, citations și LLM-as-judge;
- calibrare cu evaluare umană;
- bias checks.

Săptămâna 22

- regression gates, cost și latency budgets;
- CI care rulează eval subset;
- raport complet v1/v2.

Gate: nicio schimbare de model/prompt/chunker fără evaluare versionată.

Faza 6 — Agenți și workflows, săptămânile 23–26

Săptămâna 23

- tool schemas și structured outputs;
- trei tools read-only cu contract tests.

Săptămâna 24

- state machine, checkpoints și retry per node;
- workflow determinist.

Săptămâna 25

- dynamic routing și agent loop;
- budgets, termination și tracing;
- comparație agent versus workflow.

Săptămâna 26

- human-in-the-loop și approvals;
- tool fault injection;
- agent golden set.

Gate: agentul nu poate executa acțiuni neautorizate chiar dacă modelul cere acest lucru.

Faza 7 — Fine-tuning și text-to-SQL, săptămânile 27–31

Săptămâna 27

- Hugging Face datasets, tokenizers și Trainer;
- SFT dataset și data card.

Săptămâna 28

- LoRA/QLoRA și memory estimation;
- primul run mic și reproducibil.

Săptămâna 29

- hyperparameter experiments și MLflow;
- error analysis și overfitting checks.

Săptămâna 30

- text-to-SQL, schema linking și Spider;
- validator AST și sandbox read-only.

Săptămâna 31

- comparație base/few-shot/fine-tuned;
- serving local și load test;
- HR Assistant v1.

Gate: modelul fine-tuned trebuie să bată baseline-ul pe un set separat și să respecte toate testele de securitate.

Faza 8 — Securitate și acces, săptămânile 32–35

Săptămâna 32

- OWASP LLM Top 10 și threat model;
- atacuri directe și indirecte.

Săptămâna 33

- Llama Guard/input-output moderation;
- evaluarea false positives/negatives;
- policy versioning.

Săptămâna 34

- OAuth/OIDC, Teams SSO și Graph permissions;
- delegated access și least privilege.

Săptămâna 35

- RLS/ACL end-to-end;
- red-team suite și incident playbook;
- privacy/retention review.

Gate: access-control și adversarial tests sunt release blockers.

Faza 9 — Cloud și producție, săptămânile 36–38

Săptămâna 36

- Blob, Queue/Service Bus, managed identity și Key Vault;
- IaC pentru environment.

Săptămâna 37

- OpenTelemetry, dashboards, SLO și alerts;
- cost allocation și rate limits.

Săptămâna 38

- canary, rollback și recovery drill;
- Document Intelligence Pipeline deployed;
- runbook și postmortem simulat.

Faza 10 — Product și portofoliu, săptămânile 39–40

Săptămâna 39

- discovery, AI suitability și experiment brief;
- user testing și failure UX;
- prezentare pentru stakeholderi.

Săptămâna 40

- curățarea celor trei repository-uri;
- architecture diagrams, demos și evaluation reports;
- interviu simulat și gap analysis final.

Capitolul 15 — Checklist de readiness profesional

Software și arhitectură

- Pot proiecta un API și o schemă de date înainte de a alege framework-ul.
- Pot explica transactions, locks, idempotency și delivery semantics.
- Pot separa request path de long-running work.
- Pot testa failure modes, nu doar happy path.
- Pot livra Docker, CI/CD, IaC și observabilitate.

ML și LLM

- Pot explica train/validation/test, leakage și metricile relevante.
- Pot explica tokenizarea, attention și autoregressive decoding.
- Pot compara modele pe un dataset propriu.
- Pot estima costul și latența.
- Pot decide prompting versus RAG versus fine-tuning versus tool.

RAG și documente

- Pot păstra provenance de la răspuns la pagină și fișier.
- Pot măsura retrieval separat.
- Pot explica lexical, dense, hybrid și reranking.
- Pot aplica ACL înainte de construirea contextului.
- Pot evalua parsing, chunking și dedup.

Agenți

- Pot proiecta tool schemas și typed errors.
- Pot limita pași, timp, tokens și cost.
- Pot implementa pause/resume și approvals.
- Pot compara agentul cu un workflow determinist.
- Pot preveni ca modelul să devină security boundary.

Fine-tuning

- Pot construi și versiona un dataset.
- Pot rula și reproduce QLoRA.
- Pot diagnostica overfitting și leakage.
- Pot servi adapterul și măsura memoria/throughput.
- Pot demonstra câștigul față de baseline.

Evaluare și securitate

- Am golden set cu cazuri normale, absent, adversarial și access control.
- Am deterministic checks și evaluări probabilistice calibrate.
- Cunosc biasurile LLM-as-judge.
- Am threat model, red-team suite și incident process.
- Știu ce date ajung la provider, logs și datasets.

Stakeholders

- Pot transforma „AI poate X?” într-un experiment cu prag.
- Pot explica limitele și failure modes fără a bloca discuția.
- Pot prezenta costul total, nu doar prețul tokenilor.
- Pot propune human oversight proporțional cu riscul.
- Pot recomanda oprirea proiectului când datele nu susțin valoarea.

Capitolul 16 — Întrebări de interviu pe care trebuie să le poți rezolva

RAG

- 1 Retrieval recall a crescut, dar răspunsurile s-au înrăutățit. Cum investighezi?
- 2 De ce un reranker poate ajuta și când poate dăuna?
- 3 Cum implementezi access-controlled retrieval fără data leakage prin cache sau citations?
- 4 Cum evaluezi chunk size și overlap?
- 5 Cum tratezi documente contradictorii sau versiuni vechi?

Agenți

- 1 Când alegi workflow și când agent?
- 2 Cum previi loop-uri și cheltuieli necontrolate?
- 3 Cine autorizează un tool call?
- 4 Cum reiei execuția după crash fără efecte duplicate?
- 5 Cum testezi un agent nondeterminist?

Fine-tuning

- 1 De ce fine-tuning nu este baza de cunoștințe ideală?
- 2 Ce înseamnă rank în LoRA?
- 3 Cum construiești split-ul pentru text-to-SQL fără leakage?
- 4 De ce execution accuracy este mai utilă decât string exact?
- 5 Cum demonstrezi că modelul fine-tuned este mai bun decât few-shot?

Producție și securitate

- 1 Ce se întâmplă când mesajul din queue este livrat de două ori?
- 2 Cum urmărești o afirmație din răspuns până la document?
- 3 Cum reacționezi la indirect prompt injection într-un PDF?
- 4 De ce un guard model nu este suficient?
- 5 Ce loghezi și ce refuzi să loghezi?

Business

- 1 Ce întrebă înainte de a accepta un POC?
- 2 Cum definești un KPI pentru un research assistant?
- 3 Ce criteriu separă pilotul de producție?
- 4 Cum comunică o acuratețe de 90% când erorile au severități diferite?

Capitolul 17 — Capcane frecvente

- Să înveți numai framework-uri. Ele se schimbă; contractele, metricile și failure modes rămân.
- Să construiești cinci chatbots identice. Portofoliul trebuie să demonstreze adâncime și producție.
- Să evaluezi „cu ochiul”. Creează golden data înaintea optimizării.
- Să măsoari numai răspunsul final. Izolează parsing, retrieval, generation și tools.
- Să folosești un vector store fără baseline lexical.
- Să adaugi reranking sau query rewriting fără ablation study.
- Să aplici ACL după retrieval sau după generare.
- Să permiți modelului să execute direct SQL, shell sau API-uri mutabile.
- Să tratezi Llama Guard ca firewall perfect.
- Să salvezi prompts și documente sensibile în logs.
- Să fine-tunezi înainte să ai baseline și test set.
- Să amesteci train și evaluation data prin generare sau dedup insuficient.
- Să confunzi un notebook care rulează cu un serviciu operabil.
- Să promiți automatizare înainte să înțelegi workflow-ul uman.
- Să optimizezi calitatea fără cost, latență și risc.

Concluzie

Ținta nu este să acumulezi o listă de tehnologii, ci să dezvolti un proces repetabil:

```
problemă reală
-> date și permisiuni
-> baseline
-> prototip măsurabil
-> golden dataset
-> arhitectură sigură
-> pilot cu oameni
-> producție observabilă
-> feedback, incidente și îmbunătățire
```

Dacă poți executa acest proces pentru cele trei proiecte, inclusiv cazurile de eșec, vei acoperi competențele descrise în postare la un nivel suficient de profund pentru un rol practic de GenAI Engineer.

Capitolul 18 — Bibliografie primară și traseu de lectură

Ordinea minimă de lectură

Pentru o primă trecere, citește [S7], [S9], [S12], [S13], [S18], [S20], [S23], [S24], [S29], [S30] și documentația practică aferentă proiectului. Revino apoi la restul surselor în timpul implementării.

Software, baze de date și distribuție

[S1] Python — Coroutines and Tasks, documentație oficială.

<https://docs.python.org/3/library/asyncio-task.html>

[S2] FastAPI — documentație oficială. <https://fastapi.tiangolo.com/>

[S3] Django — Authentication and Authorization, documentație oficială.

<https://docs.djangoproject.com/en/stable/topics/auth/>

[S4] PostgreSQL — Row Security Policies, documentație oficială.

<https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

[S5] Docker — Multi-stage builds, documentație oficială.

<https://docs.docker.com/build/building/multi-stage/>

[S6] pytest — documentație oficială. <https://docs.pytest.org/en/stable/>

[S35] Azure Queue Storage — Introduction, Microsoft Learn.

<https://learn.microsoft.com/azure/storage/queues/storage-queues-introduction>

[S36] Azure Architecture Center — Competing Consumers Pattern.

<https://learn.microsoft.com/azure/architecture/patterns/competing-consumers>

ML, deep learning și LLM

[S7] scikit-learn — Model selection and evaluation.

https://scikit-learn.org/stable/model_selection.html

[S8] PyTorch — Learn the Basics. <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>

[S9] Vaswani et al. — Attention Is All You Need. <https://arxiv.org/abs/1706.03762>

[S10] Hugging Face Tokenizers — The tokenization pipeline.

<https://huggingface.co/docs/tokenizers/python/latest/pipeline.html>

[S11] Hugging Face Transformers — Cache strategies.

https://huggingface.co/docs/transformers/kv_cache

Retrieval, RAG și documente

[S12] Lewis et al. — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

<https://arxiv.org/abs/2005.11401>

[S13] Reimers și Gurevych — Sentence-BERT. <https://arxiv.org/abs/1908.10084>

[S14] PostgreSQL — Full Text Search.

<https://www.postgresql.org/docs/current/textsearch.html>

[S15] pgvector — documentația proiectului. <https://github.com/pgvector/pgvector>

[S16] GROBID — documentație oficială. <https://grobid.readthedocs.io/>

[S17] Azure Document Intelligence — Layout model.
<https://learn.microsoft.com/azure/ai-services/document-intelligence/prebuilt/layout>

Evaluare

[S18] Es et al. — RAGAS: Automated Evaluation of Retrieval Augmented Generation.
<https://arxiv.org/abs/2309.15217>

[S19] Zheng et al. — Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.
<https://arxiv.org/abs/2306.05685>

Agenți și tool use

[S20] Yao et al. — ReAct: Synergizing Reasoning and Acting in Language Models.
<https://arxiv.org/abs/2210.03629>

[S21] Schick et al. — Toolformer. <https://arxiv.org/abs/2302.04761>

[S22] LangGraph — overview, persistence și human-in-the-loop.
<https://langchain-ai.github.io/langgraph/>

Fine-tuning și serving

[S23] Hu et al. — LoRA: Low-Rank Adaptation of Large Language Models.
<https://arxiv.org/abs/2106.09685>

[S24] Dettmers et al. — QLoRA: Efficient Finetuning of Quantized LLMs.
<https://arxiv.org/abs/2305.14314>

[S25] Hugging Face PEFT — LoRA.
https://huggingface.co/docs/peft/main/conceptual_guides/lora

[S26] Hugging Face Transformers — Trainer.
<https://huggingface.co/docs/transformers/main/trainer>

[S27] vLLM — documentație oficială. <https://docs.vllm.ai/>

[S28] Yu et al. — Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Text-to-SQL. <https://arxiv.org/abs/1809.08887>

Siguranță, identitate și guvernare

[S29] OWASP — Top 10 for LLM Applications 2025.
<https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>

[S30] NIST — AI RMF: Generative Artificial Intelligence Profile, NIST AI 600-1.
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

[S31] Inan et al. — Llama Guard: LLM-based Input-Output Safeguard.
<https://arxiv.org/abs/2312.06674>

[S32] PostgreSQL — Privileges și Row Security.
<https://www.postgresql.org/docs/current/ddl-priv.html>

[S33] Microsoft Graph — Permissions overview.
<https://learn.microsoft.com/graph/permissions-overview>

[S34] Microsoft Teams — OAuth 2.0 bot authentication. <https://learn.microsoft.com/microsoftteams/platform/bots/how-to/authentication/add-authentication>

Observabilitate, MLOps și operare

[S37] OpenTelemetry — Concepts and Signals. <https://opentelemetry.io/docs/concepts/>

[S38] MLflow — Tracking. <https://mlflow.org/docs/latest/tracking/>

[S39] Kubernetes — Workloads. <https://kubernetes.io/docs/concepts/workloads/>

Product și human-centered AI

[S40] Google PAIR — People + AI Guidebook. <https://pair.withgoogle.com/guidebook-v2/>

[S41] Microsoft — Human-AI Experience Toolkit. <https://www.microsoft.com/haxtoolkit/>

[S42] Azure Well-Architected Framework — Architecture pattern for AI workloads.
<https://learn.microsoft.com/azure/well-architected/ai/architecture-pattern>